

Ex4-2-Prony-sol1

March 30, 2026

```
[326]: import numpy as np
import matplotlib.pyplot as plt
from scipy.linalg import toeplitz, solve, pinv
from scipy.signal import lfilter, freqz
```

1 Exercice 4.2.1: Approximation de Prony d'un signal a décroissance exponentielle non oscillatoire

(M.Tognolini HEIG-VD 2026 MA-StatDig)

1.1 1. Introduction et Définition du Signal

L'objectif est de tester l'approximation de Prony sur un signal test $x(n)$ [cite: 55]. Le signal est défini sur $N = 41$ échantillons par: $x(n) = [1, 3 \cdot (0.5)^n]$ pour $n=1..N-1$ avec L'objectif est de tester l'approximation de Prony sur un signal test $x(n)$. Le signal est défini sur $N=41$ échantillons par:

```
[16]: # Paramètres du signal
N = 41
n = np.arange(N)
xn = np.zeros(N)
xn[0] = 1.0
xn[1:] = 3 * (0.5)**n[1:]
print("Premiers échantillons du signal xn :")
print(xn[:8])
```

Premiers échantillons du signal xn :

```
[1.          1.5         0.75        0.375       0.1875      0.09375     0.046875
 0.0234375]
```

1.2 4.2.2. Test avec un modèle AR(2)

Nous testons un modèle purement auto-régressif (AR) avec les paramètres $p = 2$ (pôles) et $q = 0$ (zéros). La démarche est la suivante : 1. Construire la matrice de convolution X . 2. Extraire la matrice X_q et le vecteur x_{q+1} pour isoler les équations de prédiction. 3. Calculer les coefficients a_p du dénominateur en utilisant la pseudo-inverse.

```
[17]: p = 2
      q = 0

      # Construction de la matrice Xq (similaire à convmtx en Matlab)
      first_col = xn
      first_row = np.zeros(p + 1)
      first_row[0] = xn[0]
      X_full = toeplitz(first_col, first_row)

      # Extraction de Xq et xq+1 pour les équations de Prony
      Xq = X_full[q : N-1, 0:p]
      xq_1 = xn[q+1 : N]
      # Affichage de Xq et xq_1 pour vérification
      print("Matrice Xq :")
      print(Xq[0:5, :]) # Affiche les 5 premières lignes de Xq
      print("Vecteur xq_1 :")
      print(xq_1[0:5]) # Affiche les 5 premières valeurs de xq_1

      # Calcul des coefficients ap (système : Xq * ap = -xq_1)
      ap_tail = -pinv(Xq) @ xq_1
      ap = np.concatenate(([1.0], ap_tail))

      print(f"Coefficients du dénominateur ap : {ap}")
```

Matrice Xq :

```
[[1.    0.    ]
 [1.5   1.    ]
 [0.75  1.5   ]
 [0.375 0.75  ]
 [0.1875 0.375]]
```

Vecteur xq_1 :

```
[1.5    0.75    0.375  0.1875  0.09375]
```

Coefficients du dénominateur ap : [1. -1.07142857 0.42857143]

1.3 4.2.3. Calcul par les équations normales (R_x et r_x)

Pour retrouver les coefficients a_p , on peut résoudre le système lié à la matrice d'autocorrélation (ou d'autocovariance dans le cas fini) :

$$R_x \mathbf{a}_p = -\mathbf{r}_x$$

Où : * $R_x = X_q^T X_q$ est la matrice d'autocorrélation. * $\mathbf{r}_x = X_q^T \mathbf{x}_{q+1}$ est le vecteur de corrélation croisée.

```
[18]: # Calcul de la matrice Rx et du vecteur rx
      Rx = Xq.T @ Xq
      rx = Xq.T @ xq_1

      print("Matrice d'autocorrélation Rx :")
      print(Rx)
```

```

print("\nVecteur rx :")
print(rx)

# Résolution des coefficients  $a_p$  (coefficients 1 à  $p$ )
ap_tail = -solve(Rx, rx)
ap = np.concatenate([1.0], ap_tail))

print(f"\nCoefficients  $a_p$  calculés via  $R_x$  : {ap}")

```

Matrice d'autocorrélation R_x :

```

[[4. 3.]
 [3. 4.]]

```

Vecteur rx :

```

[3. 1.5]

```

Coefficients a_p calculés via R_x : [1. -1.07142857 0.42857143]

1.4 4.2.4. Calcul de la Réponse Impulsionnelle et de l'Erreur

Une fois les coefficients a_p obtenus, nous calculons la réponse impulsionnelle estimée $\hat{x}(n)$ (ou $h(n)$) et l'erreur quadratique moyenne E_{ls} . Pour un modèle AR(2), le coefficient b_0 est simplement égal à $x(0)$.

```

[19]: # Pour un AR(2),  $b_q$  est  $x(0)$  si  $q=0$ 
bq = np.array([xn[0]])

# Reconstruction du signal (Réponse impulsionnelle)
xhat = lfilter(bq, ap, np.eye(1, N)[0])

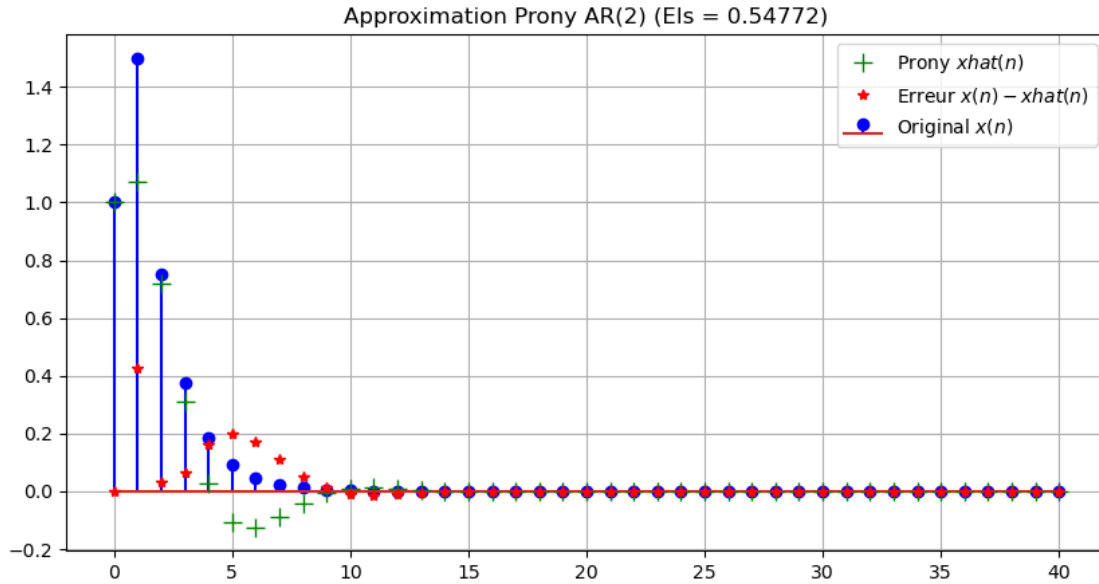
# Calcul de l'erreur
error = xn - xhat
Els = np.linalg.norm(error)
print(f"5 premier valeur de l'erreur : {error[:5]}")
print(f"Erreur Least Squares (Els) : {Els:.5f}")

plt.figure(figsize=(10, 5))
plt.stem(n, xn, linefmt='b-', markerfmt='bo', label='Original  $x(n)$ ')
plt.plot(n, xhat, 'g+', markersize=10, label='Prony  $\hat{x}(n)$ ')
plt.plot(n, error, 'r*', label='Erreur  $x(n) - \hat{x}(n)$ ')
plt.title(f'Approximation Prony AR(2) (Els = {Els:.5f})')
plt.legend()
plt.grid(True)
plt.show()

```

5 premier valeur de l'erreur : [0. 0.42857143 0.03061224 0.06341108 0.16196377]

Erreur Least Squares (Els) : 0.54772



1.5 4.2.5. Fonction Générale prony(x, p, q)

Ecrire une fonction qui automatise le processus pour tout ordre (p, q) en utilisant l'approche matricielle avec le prototype suivant:

```
def prony(x, p, q):
    ...

    return ap, bq, epq, xhat
```

Tester la fonction avec le signal précédent mais avec dans ce cas $p = 1$ et $q = 1$

```
[20]: # def prony(x, p, q):
#     N = len(x)
#     # 1. Dénominateur ap
#     X_full = toeplitz(x, np.zeros(p + 1))
#     Xq = X_full[q : N-1, 0:p]
#     xq_1 = x[q+1 : N]

#     # Utilisation des équations normales pour ap
#     Rx = Xq.T @ Xq
#     rx = Xq.T @ xq_1
#     ap = np.concatenate(([1.0], -solve(Rx, rx)))

#     # 2. Numérateur bq
#     X_start = toeplitz(x[:q+1], np.zeros(p+1))
#     bq = X_start @ ap
```

```

#     xhat = lfilter(bq, ap, np.eye(1, N)[0])
#     epq = np.linalg.norm(x - xhat)
#     return ap, bq, epq, xhat

def prony(x, p, q):
    N_sig = len(x)

    # 1. Estimation du dénominateur ap (Pôles)
    if p > 0:
        # Construction de la matrice de données décalées Xq
        X_full = toeplitz(x, np.zeros(p + 1))
        Xq = X_full[q : N_sig-1, 0:p]
        xq_1 = x[q+1 : N_sig]

        # Calcul de la matrice d'autocorrélation Rx et du vecteur rx
        Rx = Xq.T @ Xq
        rx = Xq.T @ xq_1

        # Résolution du système pour ap (coefficients 1 à p)
        ap_tail = -solve(Rx, rx)
        ap = np.concatenate([1.0], ap_tail)
    else:
        ap = np.array([1.0])

    # 2. Estimation du numérateur bq (Zéros)
    # Calculé sur les q+1 premiers échantillons
    X_start = toeplitz(x[:q+1], np.zeros(p+1))
    if p > 0:
        bq = X_start @ ap
    else:
        bq = x[:q+1]

    # 3. Reconstruction et Erreur
    xhat = lfilter(bq, ap, np.eye(1, N_sig)[0])
    epq = np.linalg.norm(x - xhat)

    return ap, bq, epq, xhat

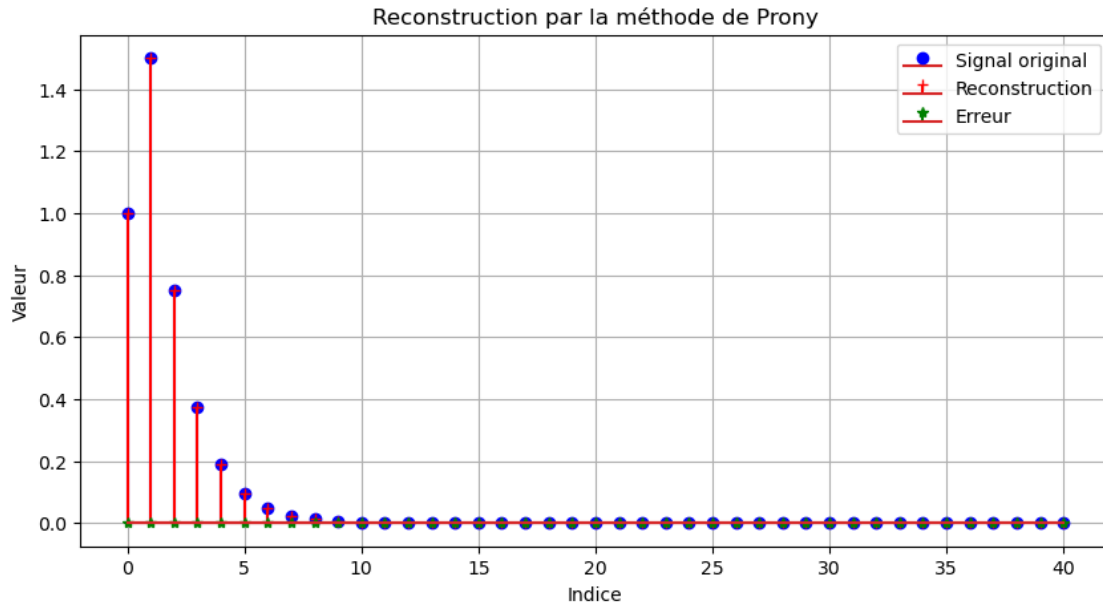
# Test final ARMA(1,1)
ap11, bq11, epq11, xhat11 = prony(xn, 1, 1)
print(f"ARMA(1,1) -> ap: {ap11}, bq: {bq11}, Erreur: {epq11:.4e}")
error11 = xn - xhat11
# plot ARMA(1,1)

plt.figure(figsize=(10, 5))
plt.stem(n, xn, linefmt='b-', markerfmt='bo', label='Signal original')
plt.stem(n, xhat11, linefmt='r-', markerfmt='r+', label='Reconstruction')

```

```
plt.stem(n, error11, linefmt='g-', markerfmt='g*', label='Erreur')
plt.xlabel('Indice')
plt.ylabel('Valeur')
plt.title('Reconstruction par la méthode de Prony')
plt.legend()
plt.grid(True)
plt.show()
```

ARMA(1,1) -> ap: [1. -0.5], bq: [1. 1.], Erreur: 0.0000e+00



2 Exercice 4.3.1 : Modélisation par la méthode de Prony de signal oscillant amorti

2.1 1. Introduction

La méthode de **Prony** est une technique de modélisation paramétrique permettant d'estimer un signal comme une combinaison linéaire d'exponentielles complexes. Contrairement à l'analyse de Fourier classique, elle permet d'estimer des fréquences qui ne sont pas nécessairement sur la grille fréquentielle, ainsi que leurs facteurs d'amortissement.

Le modèle mathématique est défini par :

$$x(n) = \sum_{i=1}^p A_i z_i^n$$

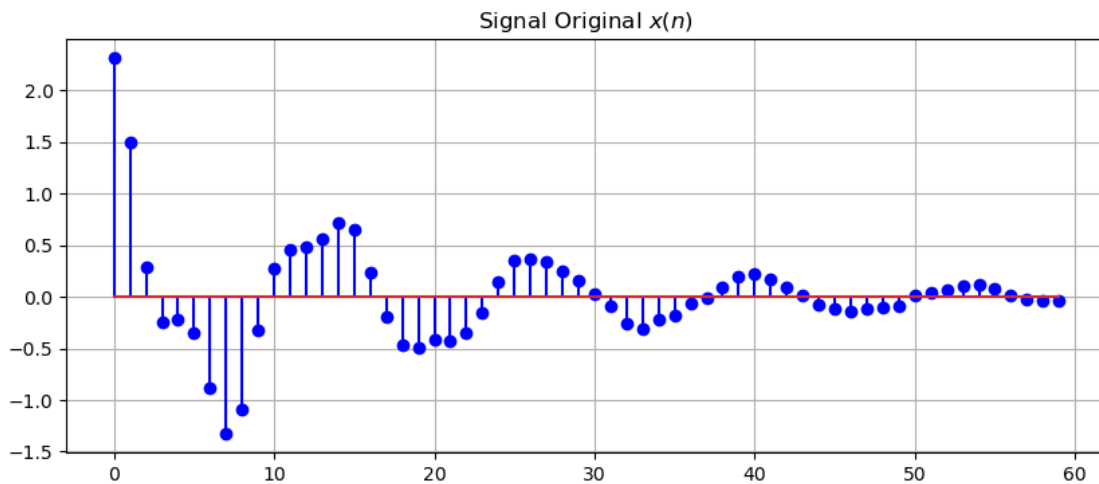
Où : * p est l'ordre du modèle (nombre de pôles). * A_i sont les amplitudes complexes (incluant la phase). * z_i sont les pôles complexes définis par $z_i = e^{s_i \Delta t}$.

```
[21]: # Fixer la seed pour la reproductibilité
np.random.seed(42)

# Paramètres
N = 60
n = np.arange(N)

# Génération d'un signal test (2 sinusoides amorties)
F1 = 0.075 # Fréquence normalisée de la première sinusoïde
F2 = 0.2   # Fréquence normalisée de la deuxième sinusoïde
x = 1.5 * (0.95**n) * np.cos(F1 * 2 * np.pi * n) + 0.8 * (0.9**n) * np.cos(F2 * 2 *
↪ * np.pi * n)
xn = x + 0.02 * np.random.randn(N) # Ajout de bruit

plt.figure(figsize=(10, 4))
plt.stem(n, xn, linefmt='b-', markerfmt='bo', label='Signal avec bruit $x(n)$')
plt.title("Signal Original $x(n)$")
plt.grid(True)
plt.show()
```



2.2 2. Étape 4.3.2 : Estimation des coefficients a_k

On suppose que le signal peut être prédit linéairement par ses échantillons passés:

$$x(n) = - \sum_{k=1}^p a_k x(n-k)$$

Pour trouver les coefficients a_k , nous résolvons le système d'équations normales au sens des moindres carrés. Nous construisons une matrice de **Toeplitz** basée sur les données observées.

On prends ici $p = 4$ et $q = 0$ car il s'agit uniquement de exponentielles décroissantes et cosinus. Le Polynôme en z du dénominateur de la fonction de transfert à les coefficients a_k que l'on trouve avec la methode prony.

```
[22]: # --- CELLULE 4 (CODE) ---
p = 4 # Ordre du modèle (2 sinusoides = 4 pôles complexes)
ap2, bq2, epq2, _ = prony(xn, 4, 4)
print(f"ARMA(2,2) -> ap: {ap2}")
print(f"bq: {bq2}, Erreur: {epq2      :.4e}")

a_coeffs = ap2

print(f"Coefficients a_k estimés : {a_coeffs}")
```

```
ARMA(2,2) -> ap: [ 1.          -2.09064816  2.30619728 -1.55751116  0.60827506]
bq: [ 2.30993428 -3.33984861  2.49775432 -0.99767857], Erreur: 1.0074e+00
Coefficients a_k estimés : [ 1.          -2.09064816  2.30619728 -1.55751116
0.60827506]
```

2.3 4.3.3. Étape 2 : Calcul des pôles complexes z_i

Les pôles du modèle sont les racines du polynôme caractéristique $A(z)$ (dénominateur de la fonction de transfert):

$$A(z) = \sum_{k=0}^p a_k z^{-k} = 0$$

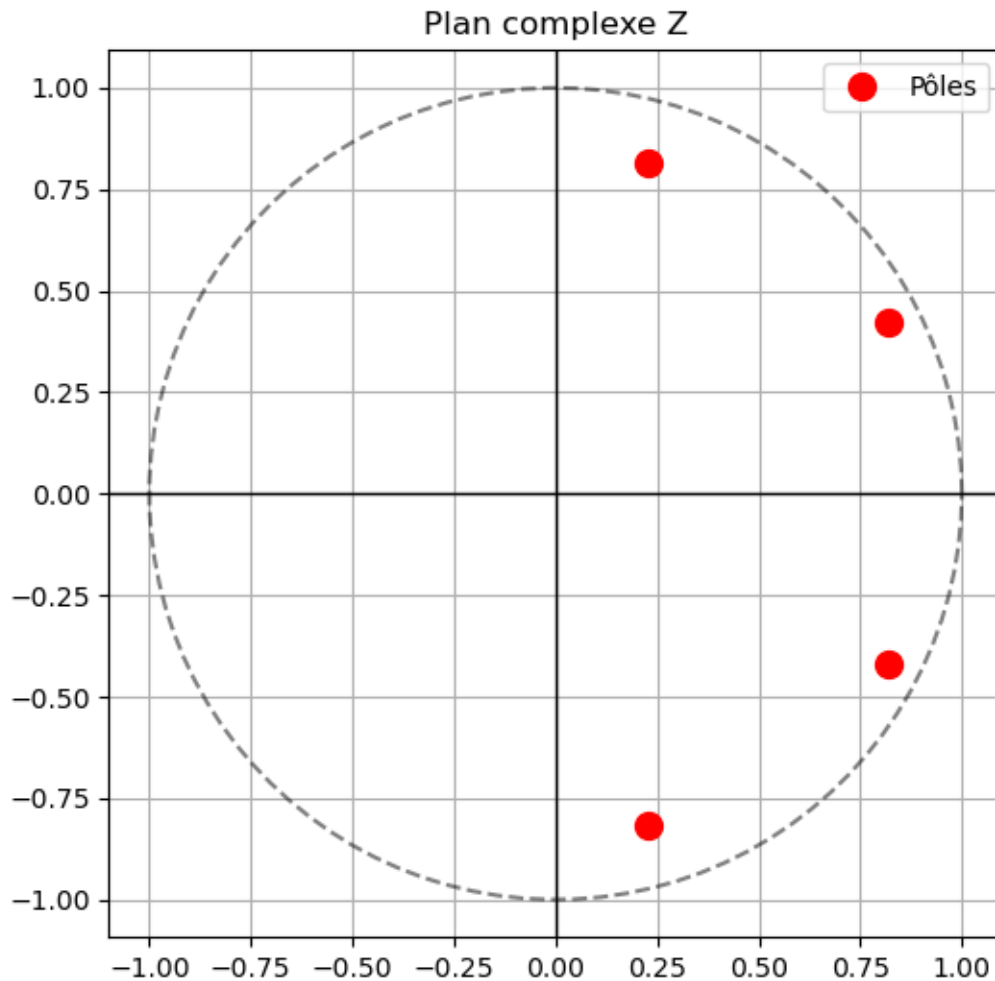
Les racines z_i nous donnent des informations sur les fréquences et l'amortissement de chaque exponentielle complexe.

```
[23]: # Calcul des racines
poles = np.roots(a_coeffs)
ModulesPoles = np.abs(poles)
PhasesPoles = np.angle(poles, deg=True)
FreqPoles = PhasesPoles / 360 # Convertir les phases en fréquences normalisées
print(f"Pôles du système en module et phase : {poles}")
print(f"Modules des pôles : {ModulesPoles}")
print(f"Phases des pôles : {PhasesPoles}")
print(f"Fréquences des pôles : {FreqPoles}")

# Affichage du plan complexe
plt.figure(figsize=(6, 6))
t = np.linspace(0, 2*np.pi, 100)
plt.plot(np.cos(t), np.sin(t), 'k--', alpha=0.5)
plt.plot(np.real(poles), np.imag(poles), 'ro', markersize=10, label="Pôles")
plt.axhline(0, color='black', lw=1)
plt.axvline(0, color='black', lw=1)
plt.title("Plan complexe Z")
plt.legend()
```

```
plt.axis('equal')
plt.grid(True)
plt.show()
```

Pôles du système en module et phase : [0.22600966+0.81556042j
0.22600966-0.81556042j 0.81931442+0.42191243j
0.81931442-0.42191243j]
Modules des pôles : [0.84629733 0.84629733 0.92156726 0.92156726]
Phases des pôles : [74.51078137 -74.51078137 27.24656068 -27.24656068]



2.4 4.3.4. Étape 3 : Estimation des amplitudes A_i

Une fois les pôles z_i connus, le signal est une fonction linéaire des amplitudes A_i . On utilise une matrice de **Vandermonde** pour résoudre le système:

$$V\mathbf{A} = \mathbf{x}$$

Où la matrice V est définie par $V_{n,i} = z_i^n$

```
[24]: # Matrice de Vandermonde
V = np.vander(poles, N, increasing=True).T
#print("Matrice de Vandermonde V :")
#print(V[:5, :]) # Affiche les 5 premières lignes de V

# Résolution pour les amplitudes complexes A_i
A_complex, _, _, _ = np.linalg.lstsq(V, x, rcond=None)
Amod = np.abs(A_complex)
Aphase = np.angle(A_complex, deg=True)

print(f"Amplitudes complexes : {A_complex}")
print(f"Modules des amplitudes : {Amod}")
print(f"Phases des amplitudes : {Aphase}")
```

```
Amplitudes complexes : [0.39935509-0.02080756j 0.39935509+0.02080756j
0.88000237-0.00538977j
0.88000237+0.00538977j]
```

3 4.3.5. Reconstruction et Résidus

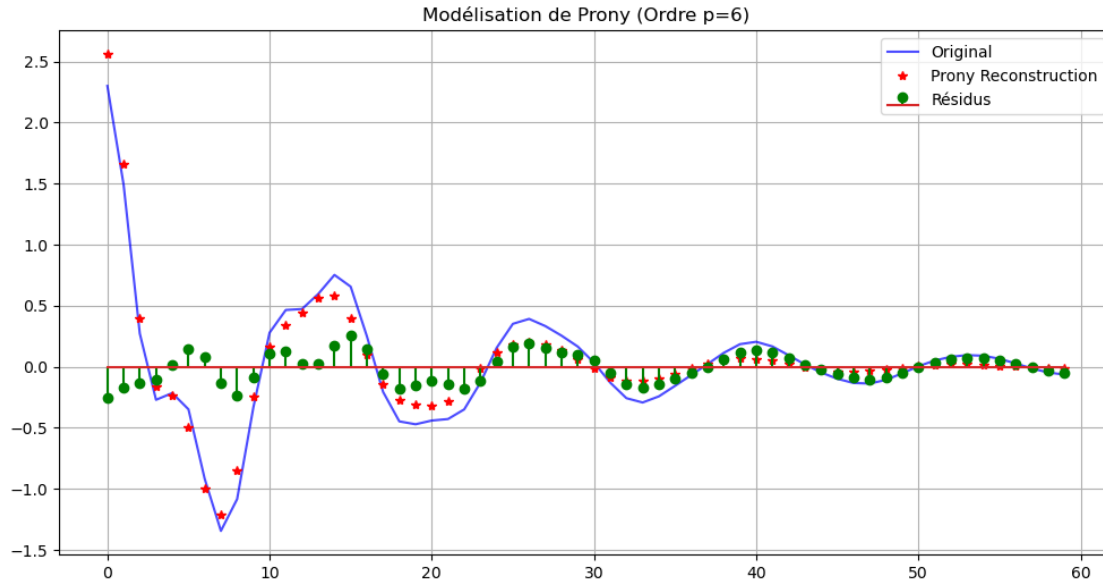
Nous reconstruisons le signal estimé $\hat{x}(n)$ à l'aide des paramètres estimés et nous calculons l'erreur de modélisation (résidu):

$$\hat{x}(n) = \text{Re} \left(\sum_{i=1}^p A_i z_i^n \right)$$

```
[25]: # Reconstruction
#x_hat = np.real(V @ A_complex)
x_hat = lfilter(bq2, ap2, np.eye(1, N)[0]) # Reconstruction avec les modules_
↳ et les coefficients ap

# Graphique de comparaison
plt.figure(figsize=(12, 6))
plt.plot(n, x, 'b--', label="Original", alpha=0.7)
plt.plot(n, x_hat, 'r*', label="Prony Reconstruction")
plt.stem(n, x - x_hat, linefmt='g-', markerfmt='go', label="Résidus")
plt.title(f"Modélisation de Prony (Ordre p={p})")
plt.legend()
plt.grid(True)
plt.show()

mse = np.mean((x - x_hat)**2)
print(f"Erreur Quadratique Moyenne (MSE) : {mse:.2e}")
```



Erreur Quadratique Moyenne (MSE) : 1.39e-02

4 Exemple 4.4.1 : Synthèse de filtre par l'approximation de Prony

4.1 1. Définition du filtre idéal

L'objectif est de synthétiser un filtre à partir de contraintes fréquentielles. On souhaite un filtre passe-bas idéal avec une phase linéaire (retard pur) :

$$H(jf) = \begin{cases} e^{-jn_d 2\pi f} & \text{pour } |f| < F_p \\ 0 & \text{pour } F_p < |f| < 0.5 \end{cases}$$

Puisque la réponse en fréquence est une fonction rectangulaire, la réponse impulsionnelle désirée $h(n)$ est une fonction **sinc** décalée de n_d échantillons pour assurer la causalité :

$$h(n) = 2F_p \text{sinc}(2F_p(n - n_d)) = \frac{\sin((n - n_d)\pi/2)}{(n - n_d)\pi}$$

```
[26]: # Paramètres du design
N = 81
Fp = 0.20
nd = 5
n = np.arange(N)

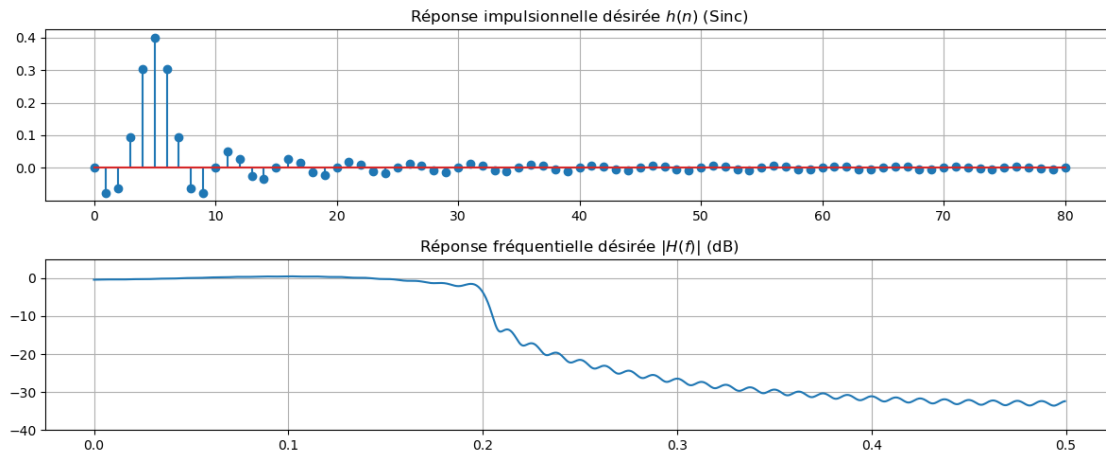
# Génération de la réponse impulsionnelle désirée (Target)
# Note: np.sinc(x) calcule sin(pi*x)/(pi*x)
hn = 2 * Fp * np.sinc(2 * Fp * (n - nd))
```

```

# Visualisation du filtre idéal
plt.figure(figsize=(12, 5))
plt.subplot(2, 1, 1)
plt.stem(n, hn)
plt.title("Réponse impulsionnelle désirée $h(n)$ (Sinc)")
plt.grid(True)

plt.subplot(2, 1, 2)
w, H_ideal = freqz(hn, 1, worN=1024)
plt.plot(w/(2*np.pi), 20*np.log10(np.abs(H_ideal)))
plt.title("Réponse fréquentielle désirée $|H(f)|$ (dB)")
plt.ylim([-40, 5])
plt.grid(True)
plt.tight_layout()
plt.show()

```



4.2 4.4.2. Fonction Prony avec équations normales

Pour estimer les paramètres du filtre ARMA (p pôles et q zéros), nous utilisons la méthode de Prony. Le dénominateur $A(z)$ est trouvé en résolvant les équations normales basées sur la matrice d'autocorrélation R_x et le vecteur r_x :

$$R_x \mathbf{a}_p = -\mathbf{r}_x$$

Avec : $* R_x = X_q^T X_q$ $* \mathbf{r}_x = X_q^T \mathbf{x}_{q+1}$

4.3 4.4.3 Methode de Shanks

La méthode de Shanks est une technique d'approximation paramétrique utilisée pour modéliser un signal par un système ARMA (Auto-Régressif à Moyenne Mobile) avec p pôles et q zéros. Elle est souvent présentée comme une amélioration de la méthode de Prony.

4.3.1 1. Le principe de la méthode

Contrairement à la méthode de Padé (qui est purement déterministe sur les premiers échantillons) ou à la méthode de Prony (qui estime le numérateur de façon simplifiée), la méthode de Shanks cherche à optimiser les coefficients du numérateur sur la globalité du signal disponible.

4.3.2 2. Les étapes de l'algorithme

La démarche se décompose en deux phases distinctes :

Estimation du dénominateur (a_p) : On commence par utiliser la méthode de Prony pour calculer les coefficients du dénominateur a_p . Ces coefficients fixent les pôles du système.

Estimation du numérateur (b_q) : C'est ici que Shanks diffère. On considère que le signal de sortie $\hat{x}(n)$ est le résultat du filtrage d'une impulsion unité par le système $H(z) = B(z)/A(z)$. * On définit une base de fonctions $g(n)$ qui correspond à la réponse impulsionnelle du filtre "tout-pôle" $1/A(z)$. * On résout ensuite un problème de moindres carrés pour trouver les coefficients b_q qui minimisent l'écart entre le signal réel $x(n)$ et la combinaison linéaire des versions décalées de $g(n)$ sur tous les échantillons.

```
[27]: def shanks(x, p, q):  
    # Dénonimateur via Prony  
    ap, _, _, _ = prony(x, p, q)  
    N_sig = len(x)  
  
    # Optimisation du numérateur bq sur tout le signal  
    # Base g(n) = réponse impulsionnelle de 1/A(z)  
    gn = lfilter([1.0], ap, np.eye(1, N_sig)[0])  
    G = toeplitz(gn, np.zeros(q + 1))  
  
    # Moindres carrés : bq = pinv(G) * x  
    bq = pinv(G) @ x  
  
    xhat = G @ bq  
    epq = np.linalg.norm(x - xhat)  
    return ap, bq, epq, xhat
```

4.4 4.4.4. Comparaison des designs : FIR vs IIR (Prony & Shanks)

Nous comparons trois approches pour approximer ce filtre passe-bas : 1. **FIR (MA)** : $p = 0, q = 10$ (Filtre à moyenne mobile). 2. **IIR Prony** : $p = 5, q = 5$ (Modèle ARMA). 3. **IIR Shanks** : $p = 5, q = 5$, où le numérateur est optimisé sur tout le signal.

```
[28]: # Calcul des trois modèles  
ap_fir, bq_fir, err_fir, x_fir = prony(hn, 0, 10)  
ap_pr, bq_pr, err_pr, x_pr = prony(hn, 5, 5)  
ap_sh, bq_sh, err_sh, x_sh = shanks(hn, 5, 5)  
  
# Affichage des erreurs L2  
print(f"Erreur FIR (q=10)      : {err_fir:.4f}")
```

```

print(f"Erreur Prony (5,5)    : {err_pr:.4f}")
print(f"Erreur Shanks (5,5)   : {err_sh:.4f}")

# Visualisation fréquentielle comparative
w, H_fir = freqz(bq_fir, ap_fir, worN=1024)
w, H_pr  = freqz(bq_pr, ap_pr, worN=1024)
w, H_sh  = freqz(bq_sh, ap_sh, worN=1024)

plt.figure(figsize=(12, 6))
plt.plot(w/(2*np.pi), 20*np.log10(np.abs(H_ideal)), 'k', label='Désiré (Sinc)',
        ↪alpha=0.5)
plt.plot(w/(2*np.pi), 20*np.log10(np.abs(H_fir)), 'r--', label='Prony FIR ↪
        ↪(q=10)')
plt.plot(w/(2*np.pi), 20*np.log10(np.abs(H_pr)), 'g', label='IIR Prony (5,5)')
plt.plot(w/(2*np.pi), 20*np.log10(np.abs(H_sh)), 'b:', label='IIR Shanks (5,5)')

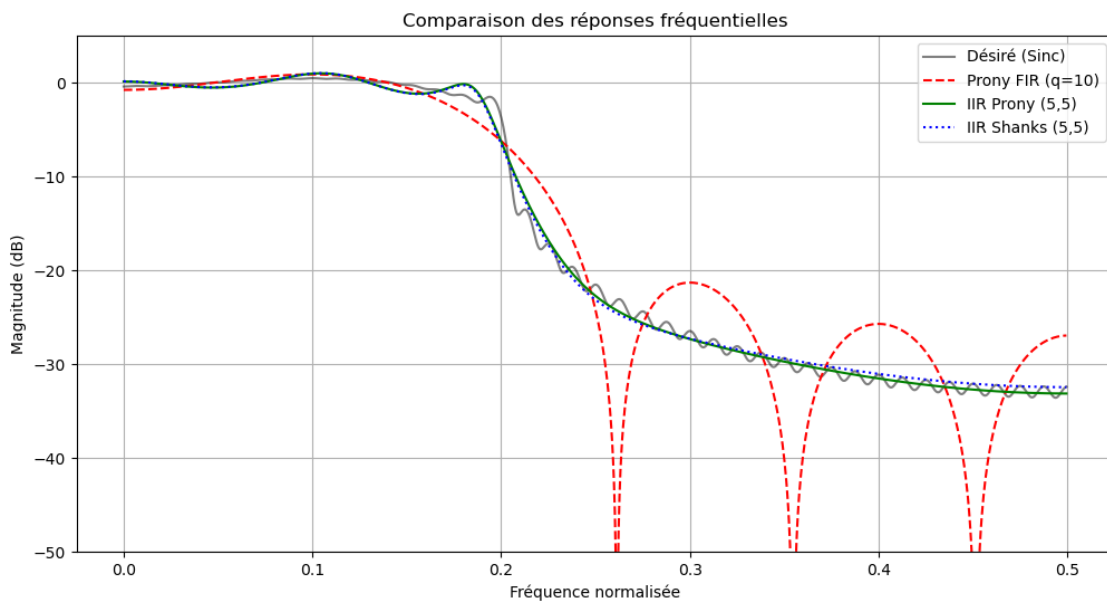
plt.title("Comparaison des réponses fréquentielles")
plt.xlabel("Fréquence normalisée")
plt.ylabel("Magnitude (dB)")
plt.ylim([-50, 5])
plt.legend()
plt.grid(True)
plt.show()

```

```

Erreur FIR (q=10)      : 0.0965
Erreur Prony (5,5)    : 0.0649
Erreur Shanks (5,5)   : 0.0645

```



4.5 4.4.5. Comparaison avec la méthode de Prony

Selon les résultats observés dans l'exercice :

- **Précision :** Pour un modèle ($p=5, q=5$), l'erreur de Shanks ($E_s=0.0645$) est légèrement plus faible que celle de Prony ($E_s=0.0649$).
- **Coefficients :** Les coefficients b_q obtenus sont légèrement différents de ceux de Prony car ils intègrent l'information de tout le signal et non seulement des $q + 1$ premiers points.
- **Stabilité :** Comme Shanks utilise les coefficients a_p issus de Prony, la stabilité du filtre (pôles à l'intérieur du cercle unité) dépend de la première étape.

En résumé, la méthode de Shanks est préférable lorsque l'on dispose d'un signal long et que l'on souhaite une fidélité maximale de la réponse impulsionnelle reconstruite $\hat{x}(n)$ par rapport au signal original.